

生成AI時代に覚えない

ユースケース記述の書き方

Yuki Yoshinaga

2025年12月29日

目次

1. イントロダクション
2. ユースケース記述とは
3. 実例：Mediumのライブラリ機能
4. 生成AIとの相性
5. まとめ

セクション1

イントロダクション

生成AI時代の開発格差

生成AIが世の中に浸透して、今ではアプリケーションを誰でも開発できるようになったと言われていますよね。

しかし、その開発速度や精度は、個々人によって大きな格差があるのが現状です。

セクション 2

ユースケース記述とは

ユースケース記述とは

物事が起きる順番に並べた箇条書き形式の作文

ユーザーとシステムとの対話を叙述的に書きます。

基本形態

ユースケース1のタイトル

- ユーザーは[XXX]する
- システムは[XXX]する
- ユーザーは[XXX]する
- システムは[XXX]する

必ず主語（「ユーザー」「システム」）を明記します。

セクション 3

実例：Mediumのライブラリ機能

2カラム比較：Before/After

Before

曖昧な指示

「記事をお気に入りにできる機能を作って」

After

ユースケース記述

- ユーザーは記事を開く
- システムは記事を表示する
- ユーザーはお気に入りボタンを押す
- システムはライブラリー一覧を表示する

2カラム対比：ユースケース記述の特徴

従来の仕様書

- 技術的詳細が多い
- 実装方法が混在
- 読み手の解釈に依存

ユースケース記述

- シンプルで直感的
- 順序が明確
- 主語が明示的

3カラムレイアウト：ユースケース記述の要素

主語

ユーザー システム

動詞

～する ～表示する

順序

時系列に沿って 箇条書きで

3カラム（アクセントカラー）

シンプル

物事を順序どおりに述べる

明確

主語を必ず書く

実用的

AIに渡すだけで実装可能

4カラムレイアウト：ユースケース記述の 利点

シンプル

理解しやすい

明確

曖昧さがない

実用的

すぐ使える

拡張性

追加しやすい

5カラム：成熟度レベル

Lv.1 基本形

Lv.2 拡張

Lv.3 応用

Lv.4 高度

Lv.5 マスター

2x2グリッド：ユースケース記述の活用 場面

開発前

要件定義 仕様書作成

開発中

実装指示 AIへの指示

開発後

ドキュメント 引き継ぎ資料

教育

新人研修 ベストプラクティス

3x2グリッド：ユースケース記述の実例

記事を読む

ユーザーは記事を開く

お気に入り

ユーザーはお気に入りボタンを押す

ライブラリ表示

システムはライブラリー一覧を表示

ライブラリ選択

ユーザーはライブラリを選択

保存

システムは記事を保存

完了

ユーザーは完了を確認

ステップリスト：記事をお気に入りにする

1. **ユーザー**はアプリケーションを開く
2. **システム**は記事一覧を表示する
3. **ユーザー**はいずれかの記事を開く
4. **システム**は当該記事を表示する
5. **ユーザー**は当該記事をお気に入りに(Save)する
6. **システム**はライブラリー一覧を表示してユーザーに尋ねる
7. **ユーザー**はライブラリを選択して完了する
8. **システム**は当該ライブラリに記事を追加する

タイムライン：開発プロセス

- 1 ユースケース記述を作成
- 2 AIに渡して実装
- 3 動作確認・修正
- 4 完成

アイコン付きリスト：ユースケース記述 の特徴



シンプル - 箇条書き形式で理解しやすい



明確 - 主語を必ず書くため曖昧さが無い



実用的 - AIに渡すだけで実装可能



効率的 - 修正指示なしで完成することも

チェックリスト：ユースケース記述のチェックポイント

- ☒ 主語（ユーザー/システム）が明記されている
- ☒ 時系列に沿って順序立てられている
- ☒ 箇条書き形式で書かれている
- ☒ 技術的詳細が含まれていない
- ☒ 読み手が理解しやすい表現になっている
- ☐ エラーケースが考慮されている（オプション）
- ☐ 代替フローが記載されている（オプション）

基本パネル

ユースケース記述の基本形

ユースケース記述は、物事が起きる順番に並べた箇条書き形式の作文です。
ユーザーとシステムとの対話を叙述的に書きます。

必ず主語を書くようにします。

強調パネル

生成AIとの相性

ユースケース記述自体は生成AI時代になって新しく登場したフレームワークではありません。
しかし、生成AIとの相性は**最高の部類**に入ります。

ガラス風パネル

ユースケース記述は「てこ」

生成AIを使った開発をより強力にしてくれる「てこ」は、これまでに培われた様々な技術やフレームワークであり、それを知り、用いることが良い生成AI開発につながります。

グラデーションパネル

実例：TODOアプリ

以前、ユースケース記述について講義をした時に、ユースケース記述からシンプルなTODOアプリをライブでバイコーディングしたことがあります。

注目すべきは、修正指示なしでアプリケーションが出来上がったことです。

ボーダーパネル

ユースケース記述の基本形態

ユースケース1のタイトル

- ユーザーは[XXX]する
- システムは[XXX]する
- ユーザーは[XXX]する
- システムは[XXX]する

これで、あるケースにおける現実世界とシステム内の挙動を端的に表すことができます。

生成AI時代の開発格差

開発速度や精度は、個々人によって大きな格差がある
経験から来る差によって、その差は広がり続けている

背景全面：画像サンプル

画像をスライド全面に配置した例です。テキストは白とシャドウで可読性を確保しています。

右側配置：実例

記事を読む・お気に入りにする

- **ユーザー**はアプリケーションを開く
- **システム**は記事一覧を表示する
- **ユーザー**はいずれかの記事を開く
- **システム**は当該記事を表示する
- **ユーザー**は当該記事をお気に入りに(Save)する

Mediumのライブラリ機能

実例として、Mediumのライブラリ機能をユースケース記述で表現しました。

左側配置：実装例

GitHubリポジトリ

<https://github.com/yoshinaga2015/TODO-App-Sample>

ユースケース記述からワンショットで作成された
TODOアプリのコードが公開されています。

実装のポイント

- ユースケース記述をAIに渡す
- 修正指示なしで完成することも
- ゼロから生成AIとアプリケーションを作る場合、必ずチャットで何往復も対話するものと思われていますが、このアプリは最初に1回「これを実装して下さい」と言っただけです。

引用スライド

生成AIにも、上手な使い方とそうでない使い方があります。

生成AIを使った開発をより強力にしてくれる「てこ」は、これまでに培われた様々な技術やフレームワークであり、それを知り、用いることが良い生成AI開発につながります。

ユースケース記述はそんな「てこ」の1つとして利用できるでしょう。

統計スライド

1

回の指示で完成

修正指示なしでアプリケーションが出来上がった

ユースケース記述は「てこ」

生成AI開発を強力にする技術の一つ

Q&Aスライド

Q. ユースケース記述は誰が書くべき？

A. デザイナーやPM、PdMでも書けます。エンジニアでなくても、シンプルな形式なので誰でも作成可能です。

Q. 技術的詳細は含めるべき？

A. 含めません。ユースケース記述は「何をするか」を記述し、「どう実装するか」は含めません。

では、この格差を一気に埋めることのできるフレームワークがあるとしたら？

デザイナーやPMがエンジニア並みに素早くアプリケーションを形にできるとしたら？

QRコード：参考資料

参考資料

- GitHub: TODO-App-Sample
- デモ: <https://yoshinaga2015.github.io/TODO-App-Sample/>
- 記事: Medium記事へのリンク

QRコード

QR Code

まとめ：要点整理

1. ユースケース記述とは

物事が起きる順番に並べた箇条書き形式の作文

2. 基本形態

主語（ユーザー/システム）を明記し、時系列で記述

3. 生成AIとの相性

最高の部類。修正指示なしで完成することも

4. 活用方法

「てこ」として、生成AI開発を強力にする

企業事例：実装例

ケース1：TODOアプリ

- ユースケース記述からワンショットで実装
- 修正指示なしで完成
- GitHubで公開中

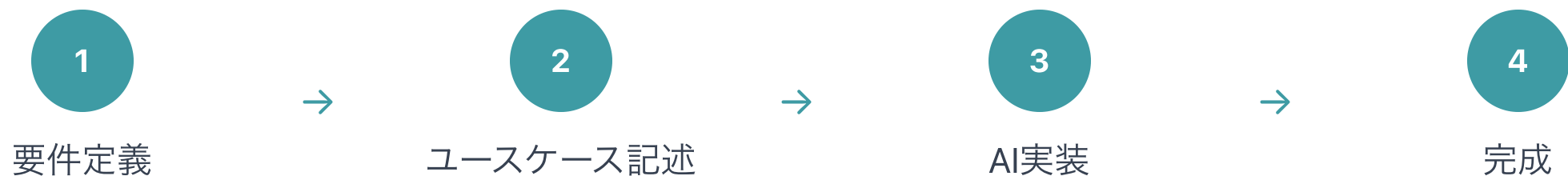
ケース2：Mediumライブラリ

- 記事をお気に入りにする機能
- ライブラリから除外する機能
- 複数のユースケースで表現

比較表：従来手法との比較

項目	従来の仕様書	ユースケース記述
形式	長文・図表	箇条書き
主語	曖昧	明示的
順序	不明確	時系列
AIとの相性	低い	高い
作成難易度	高い	低い
理解しやすさ	中程度	高い

プロセスフロー：開発フロー



メリット・デメリット

メリット

- シンプルで理解しやすい
- AIとの相性が良い
- 誰でも書ける
- 修正指示が少ない

デメリット

- エラーケースの記述が弱い
- 技術的詳細は別途必要
- 複雑なロジックには不向き

チェックポイント：ユースケース記述の確認

- ☑ 主語（ユーザー/システム）がすべての項目に含まれている
- ☑ 時系列に沿って順序立てられている
- ☑ 技術的詳細が含まれていない
- ☑ 読み手が理解しやすい表現になっている

参考資料

1. ユースケース記述からワンショットでTODOアプリをバイブコーディングする様子

- 動画デモ（無音）

2. GitHubリポジトリ

- <https://github.com/yoshinaga2015/TODO-App-Sample>

3. デモアプリ

- <https://yoshinaga2015.github.io/TODO-App-Sample/>

4. Medium記事

- 生成AI時代に覚えたい、ユースケース記述の書き方

次回予告

次回のテーマ

ユースケース記述の実践

- より複雑なケースの記述方法
- エラーケースの扱い方
- チームでの活用方法

まとめ：ユースケース記述の価値

シンプルで強力

物事が起きる順番に並べた箇条書き形式の作文

生成AIとの相性

最高の部類。修正指示なしで完成することも

誰でも使える

デザイナーやPM、PdMでも書ける「てこ」

セッション終了

まとめ

ありがとうございました

ご質問・ご意見をお待ちしています

Yuki Yoshinaga

@uxman